

Small language modeling with causal suffix copying

DASE7506 / MP1 - reproducible local project report

1. Result and objective

The frozen predictor achieves 1.66817 full-test bits per byte (BPB), compared with 2.10127 for the original 1,200-update GPT baseline (20.61% relative reduction). Development used validation only; the method was frozen before test scoring.

Evidence	Measured value
Final validation / test BPB	1.65357 / 1.66817
CPU scoring time / baseline	1.444x (limit 5x)
Peak scoring-process RAM	1.620 GiB (limit 4 GiB)
Uncompressed inference assets	10.116 MiB (limit 64 MiB)
Neural parameters / processed targets	2,222,208 / 49,152,000

2. Method

The supplied GPT provides token and learned positional embeddings, four pre-norm causal attention/MLP blocks, and tied input/output embeddings. The candidate increases width from 128 to 192 and applies 0.1 dropout to embeddings and attention/MLP residual branches. Evaluation disables dropout. All neural weights are trained from random initialization on the supplied training text.

A stateless copy component addresses local repetition. At input position t , the predictor may inspect only $x[0:t+1]$. A past context ending at j is eligible only when $j < t$, so its successor $x[j+1]$ is already observed. Among suffix lengths with at least one eligible match, the longest is used. The normalized frequency of its observed successors defines $q_t(v)$.

The final distribution is $p_t(v) = (1 - \alpha_k) p_{\text{GPT},t}(v) + \alpha_k q_t(v)$, where k is the selected suffix length. Scalar α values are selected on validation. If no eligible match exists, or copying is disabled, the exact original neural log distribution is returned. All inference memory is local to one call; no prefix state survives a window or example.

3. Data, scoring and training protocol

Protocol 7506-mp1-wt2-v2 uses the supplied WikiText-2 raw splits and fixed train-fitted BPE-2048 tokenizer. The supplied data, tokenizer, common.py, evaluate.py, model.py and baseline configuration are unchanged.

Split	Scored targets	Raw UTF-8 bytes
Validation	376,599	1,148,007
Test	428,405	1,292,013

Each target except the first token of a split is scored exactly once in independent causal windows of 256 targets, including the final short window. The BPB denominator is the entire raw split byte count. The first input of the next window is the last target of the preceding window; no hidden state carries over. Padding after a short prefix cannot influence any valid-prefix prediction.

The baseline uses seed 17, batch 32, 1,200 AdamW updates, learning rate 0.001, weight decay 0.1, a 100-step warmup, cosine decay with a 10% floor, and gradient norm clipping at 1.0. The wide main run keeps those optimizer choices and uses a 6,000-update schedule; checks and inference snapshots occur every 600 updates. Resumable chunks preserve optimizer and both sampler/dropout RNG states.

Main-run training processed 49,152,000 targets. The selected predictor retains 49,152,000 targets of ancestry. Selection rule: Uniform mean of steps 5400 and 6000, selected by minimum full-validation BPB among all main-run snapshots and two declared late averages; copy weights selected from 12 validation-only candidates.

4. Comparisons and mechanism ablation

Predictor	Training targets	Validation BPB
Original GPT baseline	9,830,400	2.07109
Original GPT + copy	9,830,400	1.97877
Wide GPT, update 1200	9,830,400	1.96000
Selected neural checkpoint (copy off)	49,152,000	1.71388
Selected final predictor (copy on)	49,152,000	1.65357

The baseline and the wide model at step 1,200 process exactly 9,830,400 targets. This is a comparison of the combined width/dropout/training-schedule design, not a clean attribution to width alone. Different full-schedule lengths also change the learning rate at that step.

The main mechanism control shares exactly the same neural checkpoint and training ancestry. With copying disabled its full-test BPB is 1.74305; with copying enabled it is 1.66817. This isolates the selected copy mechanism without extra training. The original baseline also improves on validation from 2.07109 to 1.97877 with copying alone.

5. Validation development and selection

Main-run update	Processed targets	Validation BPB
600	4,915,200	2.13493
1200	9,830,400	1.96000
1800	14,745,600	1.86791
2400	19,660,800	1.81812
3000	24,576,000	1.78991
3600	29,491,200	1.76088
4200	34,406,400	1.74256
4800	39,321,600	1.72932
5400	44,236,800	1.71965
6000	49,152,000	1.71455

Every listed checkpoint belongs to one main training trajectory, so snapshots are not independent runs. Validation checkpoints and any weight averages are compared before testing. Weight averaging, when selected, combines only self-trained snapshots and adds no training targets; ancestry records all source hashes.

Selected copy weights, from shortest to longest suffix: [0.1, 0.3, 0.5]. The initial baseline experiment considered eight candidate settings. The final search considered 12 settings, listed below. Settings are scalar hyperparameters; no evaluation answers, lookup entries, or corpus statistics are stored in the inference bundle.

Copy weights	Validation BPB
[0.0, 0.0, 0.0]	1.71388
[0.0, 0.05, 0.1]	1.67732
[0.02, 0.05, 0.15]	1.66539
[0.0, 0.1, 0.2]	1.67066
[0.02, 0.1, 0.3]	1.65937
[0.05, 0.2, 0.4]	1.65404
[0.0, 0.1, 0.2, 0.4]	1.66902
[0.0, 0.05, 0.1, 0.2]	1.67486
[0.05, 0.2, 0.4, 0.6, 0.7, 0.8]	1.65577
[0.05, 0.15, 0.3, 0.5, 0.65, 0.8]	1.65621
[0.1, 0.25, 0.45, 0.6]	1.65407
[0.1, 0.3, 0.5]	1.65357

The search is small and adaptive rather than a pre-registered statistical study. Validation is reused for model selection, so validation gains can be optimistic. The test is used only after all predictor choices are frozen, and no test-based revision is made.

6. Computation and search cost

All training used the existing Apple M5 MacBook Air (10 CPU cores, 16 GB RAM), with four PyTorch threads; no course GPU, remote GPU, pretrained weights, external training text, or rented training compute was used. CPU FP32 evaluation uses Python 3.12, PyTorch 2.7.1, NumPy 2.5.3 and tokenizers 0.21.4. Exact environment details and commands are recorded in the code bundle.

Work	Executed targets	Measured cost / note
Original baseline	9,830,400	315.42 s training
Discarded interrupted pilot	5,734,400-6,553,600	>=374.69 s training; partial work unknown
Replacement main run	49,152,000	2435.11 s training
Corpus smoke test	512	0.1112 s training; not an ancestor
Baseline copy selection	0 additional	140.21 s, 8 full-validation trials
Final copy selection	0 additional	71.39 s, 12 full-validation trials
average-last2 averaging	0 additional	0.067 s; shared ancestry
average-last3 averaging	0 additional	0.074 s; shared ancestry
Additional official validation calls	0 additional	30.21 s across 5 calls
Main-run validation checkpoints	0 additional	85.33 s across 10 calls
Baseline / pilot / smoke validation	0 additional	29.23 s; pilot from surviving log
Main-run process preparation	0 additional	46.03 s across 10 chunks

One early wide-model pilot terminated externally after the last logged update 700; no Python traceback was recorded. Its completed updates are bounded by 700-800 (5,734,400-6,553,600 targets), and at least 374.6895 training seconds are recorded. Unlogged partial work cannot be recovered. This discarded pilot is disclosed separately and does not contribute weights to the final predictor. The cause remains unconfirmed.

The main run preserves model, optimizer and sampler/dropout RNG states across process boundaries. Only final inference weights are required for evaluation. The functional smoke run adds 512 training targets to the reported search cost.

7. Resource measurements

Repeat	Baseline seconds	Final seconds
1	4.035	5.722
2	3.952	5.827
3	4.130	6.228

The median final/baseline scoring-time ratio is 1.444x. Scoring runs use the unchanged official evaluator in separate processes, CPU FP32, four threads, on the same machine after training has stopped. The table records the evaluator's scoring time; process loading/tokenization overhead is recorded separately.

The largest final-model process peak RSS is 1.620 GiB. It comes from operating-system child-process resource accounting, including Python, data loading and model inference. macOS reports `ru_maxrss` in bytes, Linux in KiB; the helper converts accordingly. The CPU evaluator's zero GPU-allocation field is not used as a RAM measurement. Inference assets total 10,607,623 uncompressed bytes, including the checkpoint, predictor source, baseline model source and tokenizer.

8. Critical analysis

Exact suffix copying can help repeated names, phrases and spelling patterns within the visible window. Its evidence is directly interpretable: it reallocates probability to successors already observed after the same suffix. Longer suffixes reduce accidental matches but occur less often. Short suffixes can match unrelated contexts, and an aggressive mixture can lower the probability of the correct continuation; this is why the copy weight needs validation selection.

The method has no memory across the independent 256-token windows. It cannot retrieve a name from an earlier window, and it has no large training-derived retrieval index. For contexts without matches, performance is entirely determined by the neural model. A stronger neural model may reduce the marginal gain from copying; the same-weight ablation measures the actual remaining benefit for this predictor.

For a window of T tokens and K suffix orders, copying performs $O(K T^2)$ equality/mask work per example and uses transient $O(T^2 + T V)$ storage, including the output distribution over $V=2048$ tokens. Its cost depends on the fixed window and batch sizes, not the size of the training corpus. The extra probability mixing and suffix scans trade scoring time for local repetition accuracy.

The experiments use one training seed. They establish a reproducible result and a paired mechanism comparison for this run, not a distribution over training seeds. Changes in capacity, dropout and total schedule are bundled, so attributing the combined improvement to one architectural factor would overstate the evidence. Additional seeds and separately controlled regularization/capacity experiments would be necessary for broader claims.

Validation checkpoint/mixture selection can overfit development decisions. All trial results and discarded-work costs are recorded in results.json, and the untouched test is scored only after freezing. Repeated test evaluations measure reproducibility and timing of an unchanged predictor; they are not additional tuning opportunities. CPU timing and floating-point details vary with hardware and libraries, so the actual raw measurements and pinned environment accompany the claimed resource ratio.

9. Integrity, authorship and AI assistance

The official classroom baseline is reused with attribution. OpenAI Codex substantially assisted with interpreting the requirements, designing and implementing the suffix-copy model, writing tests and experiment tools, executing local training and validation, checking sources, and drafting this report and reproduction documents. The student is responsible for understanding the implementation.

The unchanged supplied WikiText-2 text/tokenizer and hash manifests accompany the code as allowed by the assignment, with the original Wikipedia/WikiText attribution notices retained.

References

[1] DASE7506. MP1 Small Language Model Challenge: supplied GUIDE.md, model.py, common.py, evaluate.py and code README, 2026. Assignment protocol 7506-mp1-wt2-v2.

[2] Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer Sentinel Mixture Models. 2016. arXiv:1609.07843. <https://doi.org/10.48550/arXiv.1609.07843>

[3] Grave, E., Joulin, A., and Usunier, N. Improving Neural Language Models with a Continuous Cache. 2016. arXiv:1612.04426. <https://doi.org/10.48550/arXiv.1612.04426>

References [2-3] motivate recent-history mixtures; this implementation uses exact suffix counts and does not reproduce their neural pointer/cache architectures or claim their published scores.